reduce/binomial

An AgentMPI collective scaling analysis

Abstract

The binomial reduction is AgentMPI's default for $p > 3$ and the dual of the binomial broadcast: the same tree, the same $p-1$ messages, the same $\lceil \log_2 p \rceil$ rounds, run inward instead of outward. Across 21 measured sizes from $p = 2$ to $p = 32$ it logged exactly $p-1$ messages, reported that count exactly, recorded $\lceil \log_2 p \rceil$ rounds, and — the number only a reduction has — recorded a `fold_depth` of exactly $\lceil \log_2 p \rceil$: $1, 2, 2, 3, 3, 3, 3, 4, 4, 4, 5, 5, 5$ across the thirteen sizes, agreeing at 21 of 21 with no remainder-path defect and no red crosses. Setting that against the broadcast is what makes the pair worth analysing together, and the trace states the difference without interpretation: the broadcast's $p-1$ messages carry exactly *one* distinct content digest at every size, while this reduction's $p-1$ messages carry $p-1$ *distinct* digests at every size — even though all p ranks contributed the identical value 1 under `SUM`. A broadcast forwards a handle; a reduction manufactures a new artifact at every hop. The single most important thing to take away is that this is the algorithm that makes the two axes cheap simultaneously: it holds traffic at the flat reduction's $p-1$ while cutting the number of *successive* operator applications from $p-1$ to $\lceil \log_2 p \rceil$ — five instead of thirty-one at $p = 32$ — and archived agent-executed runs at $p = 8$ show that buying a 2.5-fold latency win over `flat` and an 8-to-17-fold win over `chain`. I also report a minor accounting defect: the collective's `tokens_sent` under-reports the traffic the fabric logged by a fixed 19 tokens per message, because the tracker charges the accumulator and the transport moves a four-field envelope.

1 What this family is

The `reduce` collective under the `binomial` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.012–0.104 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

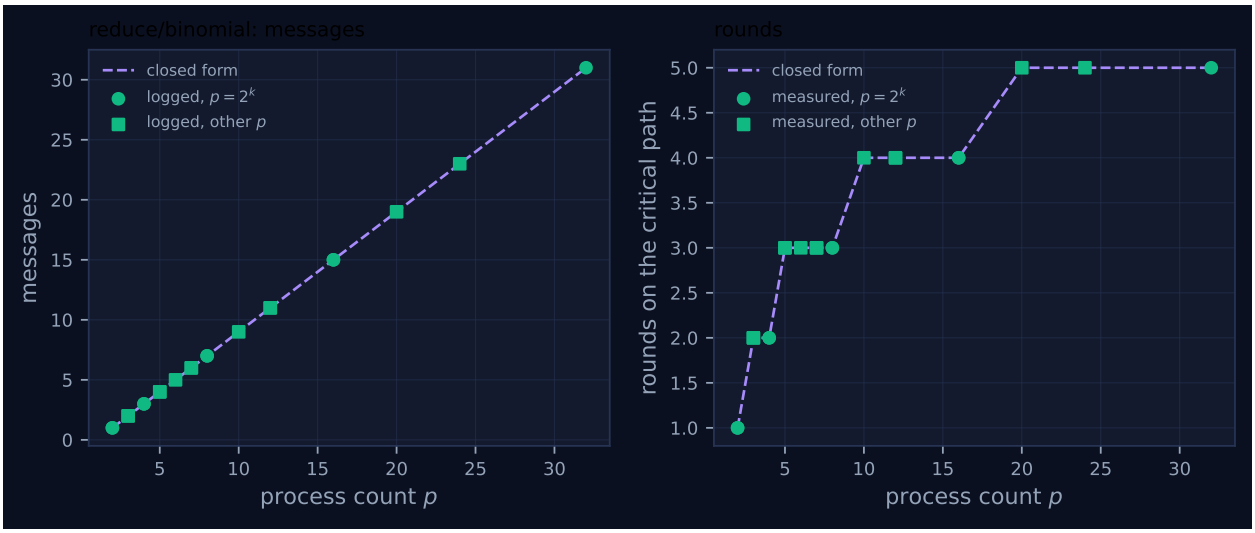

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	1	1	1	1	1	0.012	✓	✓
2	`mb-smoke`	1	1	1	1	1	0.013	✓	✓
3	`mb-free`	2	2	2	2	2	0.012	✓	✓
3	`mb-smoke`	2	2	2	2	2	0.013	✓	✓
4	`mb-free`	3	3	3	2	2	0.018	✓	✓
4	`mb-smoke`	3	3	3	2	2	0.023	✓	✓
5	`mb-free`	4	4	4	3	3	0.039	✓	✓
5	`mb-smoke`	4	4	4	3	3	0.024	✓	✓
6	`mb-free`	5	5	5	3	3	0.026	✓	✓
7	`mb-free`	6	6	6	3	3	0.054	✓	✓
7	`mb-smoke`	6	6	6	3	3	0.019	✓	✓
8	`mb-free`	7	7	7	3	3	0.031	✓	✓
8	`mb-smoke`	7	7	7	3	3	0.026	✓	✓
10	`mb-free`	9	9	9	4	4	0.058	✓	✓
12	`mb-free`	11	11	11	4	4	0.048	✓	✓
12	`mb-smoke`	11	11	11	4	4	0.049	✓	✓
16	`mb-free`	15	15	15	4	4	0.082	✓	✓
16	`mb-smoke`	15	15	15	4	4	0.062	✓	✓
20	`mb-free`	19	19	19	5	5	0.082	✓	✓
24	`mb-free`	23	23	23	5	5	0.079	✓	✓
32	`mb-free`	31	31	31	5	5	0.104	✓	✓

4 How the algorithm scales

The implementation is the binomial broadcast tree traversed in reverse. Each rank computes a virtual rank $vr = (r - root) \bmod p$ and scans masks $1, 2, 4, \ldots$ upward. On each mask, if vr has that bit set it sends its accumulator to $parent = vr - mask$ and stops; otherwise, if $vr \mid mask < p$ it

receives from that child and combines. The combine is deliberately ordered:

```
acc = op.fn(acc, got["acc"], ...)
```

with the local accumulator as the *left* operand, so the lower virtual rank is always the accumulator and rank order is preserved along every path. Each rank except $vr = 0$ sends exactly once, giving $p - 1$ messages; the mask loop runs $\lceil \log_2 p \rceil$ times, giving $\lceil \log_2 p \rceil$ rounds. That is `FORMULAS[("reduce","binomial")]`, which records $(\lceil \log_2 p \rceil,\ p - 1,\ (p-1)n,\ \lceil \log_2 p \rceil)$.

The fourth term is the one worth deriving carefully, because `fold_depth` is not the tree's path length and the sweep is what makes the difference concrete. The implementation accumulates it as `depth = max(depth, got["depth"]) + 1` on every receive, so it counts *successive operator applications* on the longest chain from a leaf to the result — which is the fidelity-relevant quantity, since each application is another transformation of whatever came before. Take $p = 7$. The tree is $0 \to \{4, 2, 1\}$, $4 \to \{6, 5\}$, $2 \to \{3\}$, so the longest path is two hops. But rank 4 applies the operator twice (once with rank 5's contribution, once with rank 6's) and rank 0 applies it three times (children 1, 2, then 4), and because the accumulator is the left operand each application re-encodes the result of the last. Rank 5's contribution therefore passes through three transformations, not two, and the measured `fold_depth` at the root of `mb-free__coll-reduce-binomial-7` is 3. This is why the mirrored broadcast at $p = 7$ has a maximum `tree_depth` of 2 while the reduction has a `fold_depth` of 3: a broadcast's cost is how far the payload travelled, a reduction's is how many times it was rewritten, and at non-powers of two those are different integers.

The traversal is visible event by event. In `mb-free__coll-reduce-binomial-8` the leaves $1, 5, 3, 7$ send at 6.42, 10.07, 13.19 and 15.31 ms, each reporting `fold_depth: 0` because it applies nothing. Rank 6 receives from 7 and forwards at 20.02 ms with `fold_depth: 1`; rank 2 receives from 3 and forwards at 21.74 ms with `fold_depth: 1`; rank 4 receives from 5 and then from 6 and forwards at 30.30 ms with `fold_depth: 2`; rank 0 receives from 1, 2 and 4 and records `fold_depth: 3` at 30.96 ms. Three levels, seven messages, 25.3 ms of root blocking.

Figure 1 is the two axes side by side. The left panel is the straight line $p - 1$ with every marker on it — identical to `flat`'s and `chain`'s left panel, which is the premise of the comparison. The right panel is the logarithmic staircase, with treads at 1 ($p = 2$), 2 ($p = 3, 4$), 3 ($p = 5, 6, 7, 8$), 4 ($p = 10, 12, 16$) and 5 ($p = 20, 24, 32$). The risers sit at $p = 2^k + 1$, where $\lceil \log_2 p \rceil$ increments, and the sweep brackets them rather than pinning them: it contains 8 and 10 but no 9, and 16 and 20 but no 17. Every measured point lies on the correct tread and the points immediately either side of each riser are measured, which is as strong a statement as the available sizes support. Because `fold_depth` equals the round count for this algorithm, the same staircase is also the fidelity-relevant curve — which is not true of `flat`, where the round count is flat at 1 and the fold depth climbs to 31.

The viewer screenshot for `mb-free__coll-reduce-binomial-32` shows the work distributed rather than concentrated. Message glyphs appear in almost every lane, and the lanes that carry several are the interior nodes of the tree; rank 0's glyphs are late (roughly 0.04, 0.08 and 0.105 s) because a root receives last, and rank 16 — the top-level child — carries a tick at about 0.10 s just before the root closes. The stats row reports 31 messages, 0 agent calls and \$0.000 over a 0.1 s span. Compared with `mb-free__coll-reduce-flat-32`, where rank 0's lane alone carries a long spread of glyphs, this is the same 31 messages arranged so that no rank does more than $\lceil \log_2 p \rceil$ of the work.

5 Powers of two and everything else

The non-power-of-two path is a single guard, `if child_v < p`, which suppresses receives from virtual ranks that do not exist. That is the entire remainder handling, and it behaves. All eight non-power-of-two sizes logged $p - 1$ messages (2 at $p = 3$, 4 at $p = 5$, 5 at $p = 6$, 6 at $p = 7$, 9 at $p = 10$, 11 at $p = 12$, 19 at $p = 20$, 23 at $p = 24$), self-reported the same figure the fabric logged, and recorded rounds and fold depth both equal to $\lceil \log_2 p \rceil$. There are no red crosses in Figure 1.

Two remarks about the remainder sizes, one reassuring and one a genuine constraint.

The reassuring one: because `fold_depth` counts applications rather than hops, it is $\lceil \log_2 p \rceil$ even at the sizes where the tree is one level shallower than that. At $p = 7$, $p = 12$ and $p = 24$ the maximum path length in the mirrored broadcast is 2, 3 and 4 while the reduction's fold depth is 3, 4 and 5. A reader who assumed the reduction's fold depth tracked the broadcast's tree depth would under-count the re-encodings at exactly the non-power-of-two sizes, which are the common case for an agent population. The implementation recording the two quantities under different names in different collectives is what prevents that error, and the family view is where the discrepancy is visible.

The constraint: a binomial reduce is *refused* for a non-commutative operator at a non-zero root. The tree keeps the lower virtual rank as the accumulator, so evaluation order is virtual rank order — which coincides with communicator rank order only when the root is 0. For any other root the implementation raises `AmpiUsageError` rather than silently evaluating in rotated order, and directs the caller to `chain` or `flat`. This is a real restriction on the algorithm rather than a bug, and it is the analogue of MPI refusing to reorder an operator declared non-commutative, made stricter. None of the 21 runs in this sweep exercises it: every one used `SUM` (commutative, EXACT) with `root = 0`, so the guard is untouched and my account of it rests on the code and on `test_reduce_preserves_order_for_noncommutative_op`, which checks `CONCAT` against the reference fold at root 0 across nine sizes.

The guard is also, as far as I can tell, correct and unique. Its condition names `binomial` specifically, and `kary` — which performs the identical virtual-rank rotation — is not covered. A four-rank probe with `CONCAT` at `root = 1` has `binomial` raising `AmpiUsageError` as designed while `kary` returns `'[1][2][3][0]'` in place of `'[0][1][2][3]'`, silently. That is reported in the `reduce/kary` analysis; it is worth noting here because it means this algorithm's refusal is the correct behaviour and its wide-arity sibling is the outlier, not the reverse.

A minor accounting defect: token volume is under-reported by a fixed envelope

Message counts agree everywhere, but token volumes do not. Summing the `tokens` field of the `msg.send` events gives $20(p - 1)$ at every size — 140 at $p = 8$, 300 at $p = 16$, 620 at $p = 32$ — while the collective reports $p - 1$ as `tokens_sent`: 7, 15 and 31. The cause is a one-line mismatch. The algorithm transmits a four-field envelope, `{"acc": ..., "depth": ..., "weight": ..., "vr": ...}`, and the tracker charges `tr.sent(_tok(comm, acc))` — the accumulator alone. The difference is a fixed additive constant per message, 19 tokens here and (measured with `tokens.count` on string payloads from 1 to 8,200 tokens) 21 tokens independent of payload size.

This is bounded and it is not the same class of error as a wrong message count: at a realistic 1,000-token artifact the under-report is about 2%, and at a one-token scalar it is 95% only because the denominator is one token. But the direction is the one the repository's own test suite says a cost model must never have — the accounting test's docstring notes that "an under-reporting collective looks cheaper than it is" — and no test covers the token axis:

`test_self_reported_message_count_equals_the_traffic_actually_logged` compares `messages_sent` against `msg.send` counts and does not compare `tokens_sent` against the logged `tokens`. `reduce/flat` has no such discrepancy, because it sends the payload bare; `reduce/chain` has the same one at 15 tokens per message, its envelope having three fields rather than four. I would call this a real if small defect, and the cheap fix is to charge `_tok` of the envelope that is actually sent.

Eight sizes were measured twice, under `mb-free` and `mb-smoke`. Every count — messages logged, messages reported, rounds, tokens, participants — is identical at all eight; run wall times differ by factors from 1.01 to 2.82, the largest at $p = 7$ where the absolute figures are 0.054 and 0.019 s. Two independent measurements agreeing on every integer and disagreeing on every duration is the correct summary of the sweep's competence.

6 When to choose this algorithm

For an associative operator, this is the default and the sweep supports it. All three of `flat`, `binomial` and `chain` send $p - 1$ messages and perform $p - 1$ operator applications, so they differ only in the arrangement, and binomial's arrangement is the only one of the three that is short on both the round axis and the fold axis: $\lceil \log_2 p \rceil$ rather than 1 and $p - 1$ (flat) or $p - 1$ and $p - 1$ (chain). At $p = 32$ that is 5 against 31 successive applications, and 5 against 31 rounds.

The measured consequence, from the archived agent-executed runs at $p = 8$, is unambiguous. Root blocking inside the reduction was 77.6 s and 80.3 s under `binomial`, against 197.6 and 251.6 s under `flat` and 1,377.0 and 634.4 s under `chain`; output tokens were 2,903 and 2,492 against flat's 3,456 and 3,372 and chain's 3,855 and 2,623. So binomial is 2.5–3.1 times faster than flat and 8–17 times faster than chain at the same message count and the same number of applications, purely because it runs those applications concurrently across levels. That is the whole argument for logarithmic depth in an agent setting, and it is why the ordering here is not the ordering the network-round count would suggest — flat has one round and is nearly three times slower.

Where binomial is beaten, it is beaten by `kary`. A k-ary tree with a variadic operator sends the same $p - 1$ messages but combines all k children in one application, so its fold depth is $\lceil \log_k p \rceil$ and its *application count* is $\lceil (p - 1)/(k - 1) \rceil$ rather than $p - 1$. Measured at $p = 8$ with $k = 4$: three `agent.call` events against binomial's seven, fold depth 2 against 3, and 53.1 and 51.8 s of root blocking against 77.6 and 80.3. Binomial is the right default only because it needs no variadic kernel; where one exists, the binary arity is a hardware assumption — single-ported processes — that agent ranks do not satisfy.

On fidelity the honest position is more interesting than the intuition. Structurally, cutting $p - 1$ successive re-encodings to $\lceil \log_2 p \rceil$ should preserve more of a lossy operator's input, and the cost model's $F(d) = (1 - \delta)^d$ makes that concrete: 0.774 for binomial against 0.204 for flat and chain at $p = 32$ with $\delta = 0.05$. The re-encoding is demonstrably real — $p - 1$ distinct digests per run, against the broadcast's one. But this repository's fidelity experiment measured retention at 0.3854 under all four algorithms at $p = 8$ (`inc-mb-fidelity-inc`, 37 of 96 items surviving at fold depth 7, 3 and 2 alike), and at 1.0000 under all four when the output budget was not binding (`sat-mb-fidelity`). Under a uniform enforced budget the final application is the binding one, so a deep tree discards at intermediate levels and a shallow one discards at the root and both arrive at the same retention. The case for binomial over flat and chain is therefore cost — latency and output tokens, both measured above — and the fidelity argument is a prediction the harness's own measurement did not confirm. Depth still *could* matter under a subtree-proportional budget, which is what the operator

interface's `weight` field exists to enable, but no archived run uses it.

7 Threats to this reading

These runs contain no agents. All 21 report `executors = {"none": p}`, zero `agent.call` events, zero busy time, an idle fraction of 1.0 and $0.000 of cost, and the operator was `SUM` over a one-token scalar, so every one of the $p - 1$ applications was an integer addition. The quantity that makes the depth argument matter — the cost and the lossiness of an operator application — is exactly zero here. What this sweep establishes is that the implementation's message count, round count and fold depth are the integers the closed form predicts at 21 of 21 sizes, from two independent sources (the algorithm's own bookkeeping and the fabric's transport log). Every latency and fidelity figure I quote in the previous section comes from four single-point runs at $p = 8$ and $p = 16$, not from this sweep, and two of those four use a surrogate function operator rather than a model.

The wall times at large p do not measure the collective. In `mb-free__coll-reduce-binomial-32` the thirty-two `rank.init` events span 1.2 to 92.1 ms while the whole run lasts 104.2 ms: the last rank started as the reduction was finishing. The 0.104 s in the generated table and the 84.7 ms spread between the first and last `coll.reduce` record are launch stagger, not synchronisation cost. This also explains why the per-round cost in this family is non-monotone — 20.0 ms at $p = 16$, 14.0 at $p = 24$, 20.4 at $p = 32$ — rather than reflecting anything about the tree.

Several code paths are unexercised. Every run used `root = 0`, so the rotation $vr = (r - root) \bmod p$ is never driven with a non-zero root, and in particular the `AmpiUsageError` guard that refuses a non-commutative operator at a non-zero root is never reached — which is unfortunate, because that guard is the most consequential correctness property this algorithm has. Every run used a commutative, EXACT operator, so the ordered left-operand discipline that makes the tree legal for `CONCAT` is untested here. Every run used a one-token payload, so the volume term $(p - 1)n$ is validated only at $n = 1$ and the envelope under-report I report above is measured only where the envelope dominates. And zero of 1,110 receptions across the six sweeps admitted a token into a rank's context, so the context pressure that would decide between a binary tree and a wide one is not exercised at all.

Finally, the `fold_depth`-versus-`tree_depth` argument compares two numbers produced by two different collectives on two different runs, not two measurements of the same event. It is a statement about what the two implementations record and why their conventions differ, supported at 21 of 21 sizes in each family; it is not a measurement of how many times any particular token was rewritten, which with `SUM` over scalars would not be observable even in principle.